58

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

CI ile GitOps Köprüsü: GitHub Actions'ın Flux Deposunu Güncellemesi

Bridging CI to GitOps: GitHub Actions Updating Flux Manifests

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

İki muazzam dünyayı birleştiren altın köprü! GitHub Actions (CI) derlemeyi bitirdiğinde doğrudan gidip K8s kümesine saldırmaz (Bu GitOps kuralına aykırıdır). Bunun yerine gider, Flux'ın izlediği GitOps filosunun deposundaki YAML dosyasını açar: 'İmaj etiketini az önce derlediğim yeni SHA ile güncelledim' diyerek bir commit atar. Ve sahneden çekilir; gerisini Flux halleder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Golden Bridge connecting two mighty engineering realms! When GitHub Actions (CI) finishes building, it NEVER attacks the Kubernetes cluster directly (that violates GitOps!). Instead, it walks over to the separate GitOps repository tracked by Flux, updates the image tag inside `deployment.yaml` with the fresh commit SHA, and pushes a clean commit. CI exits; Flux takes the wheel!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kurumsal ayrım prensibi (Separation of Concerns): 1) Kod Reposu (Uygulama kodu + CI pipeline). 2) GitOps Reposu (K8s manifestoları + Ortam ayarları). GitHub Actions derlemeyi bitirdiğinde: `actions/checkout` ile GitOps reposunu klonlar, `clusters/production/.../deployment.yaml` içindeki imaj etiketini yeni derlenen `image: ghcr.io/...:${{ github.sha }}` ile günceller (sed veya Kustomize edit ile), ve GitOps reposuna bir 'chore(release)' commit'i atar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Separation of concerns: 1) App Repo (source code + test pipelines), 2) GitOps Fleet Repo (Kubernetes manifests + infrastructure state). At the tail of the CI run, the workflow clones the fleet repo, updates the target deployment manifest with the newly pushed container SHA via `kustomize edit set image` or `sed`, and commits back to the fleet repo, triggering Flux in-cluster.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# GitHub Actions'tan GitOps Reposuna Otomatik Terfi (Promotion):
- name: GitOps Deposunu Klonla
  uses: actions/checkout@v4
  with:
    repository: 'my-org/k8s-gitops-fleet'
    token: ${{ secrets.GITOPS_DEPLOY_PAT }}
    path: fleet-repo

- name: İmaj Etiketini Güncelle ve Push Et
  run: |
    cd fleet-repo/clusters/production/coffee-app
    sed -i "s|image: .*|image: ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}|
g" deployment.yaml
    git config user.name "github-actions[bot]"
    git config user.email "actions@github.com"
    git add deployment.yaml
    git commit -m "chore(release): promote coffee-app to version ${{ github.sha }}"
    git push origin main
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Uygulama kodunun olduğu repo ile GitOps manifestolarının olduğu repoyu ayrı tutmak en iyi kurumsal pratiktir. Böylece her Java kod değişikliğinde sonsuz bir CI/CD döngüsü (infinite trigger loop) yaşanması engellenir.

🇬🇧 WATCH OUT

Decoupling application repositories from GitOps fleet repositories prevents catastrophic infinite pipeline trigger recursion loops.

🎯 Özet Çıkarımlar

- CI altyapısı hiçbir zaman doğrudan Kubernetes kümesine bağlanmaz.
- Dağıtım tetikleyicisi Git üzerindeki temiz bir commit'e dönüşür.
- Tek bir `git revert` komutuyla tüm üretim kümesi önceki sürüme geri döner.

🎯 Key Takeaways

- CI build runners require zero direct network access or credentials to production clusters.
- The release event is formalized as a transparent Git commit in the fleet repo.
- Rolling back production is as simple and safe as executing `git revert` on the fleet repo.